CLIP, VLMs & LLaVA

A mathematical and conceptual study guide | October 2026

Audience: readers familiar with Vision Transformers, decoder-only Transformers, PyTorch, and introductory deep learning. Equations use plain-text notation for clarity.

1. The big picture: what is a VLM?

A vision-language model (VLM) is any model that learns relationships between visual information and natural language. It is a broad family, not one architecture. Some VLMs align image and text representations (CLIP); others generate captions or conversational answers (BLIP, LLaVA). Image generation is a separate capability and is not implied by the label VLM.

Model	Typical input	Typical output	Primary role
ViT classifier	Image	Class logits	Fixed-label recognition
CLIP	Image and/or candidate text	Embeddings, similarities	Retrieval, zero-shot classification
LLaVA	Image + instruction	Generated text	Captioning, visual Q&A
Text-to-image diffusion	Text prompt	Generated image	Image synthesis

2. Why vision-language learning?

Motivation A - beyond fixed labels. Traditional classifiers use predefined classes and need new labeled data to add concepts. Natural language lets users describe arbitrary candidate concepts, enabling open-vocabulary or zero-shot recognition.

Motivation B - scalable supervision. Image-caption pairs provide richer and more abundant training signals than single class IDs. CLIP showed how contrastive pretraining on 400 million web-collected image-text pairs can transfer to many downstream tasks [1].

Motivation C - communicate about what is seen. Generative VLMs answer questions, summarize charts, and describe scenes in sentences. They connect visual encoders to language decoders, but may hallucinate or misread fine detail.

Historical context. Image captioning research predates CLIP, including Show and Tell (2015) [4]. CLIP (2021) was a major milestone for contrastive image-language alignment, while LLaVA (2023) emphasized visual instruction following [1,3].

3. CLIP: architecture and representations

CLIP = Contrastive Language-Image Pre-training. The original model jointly trains an image encoder (ResNet or ViT) and a text Transformer, with trainable projections into a common D-dimensional space [1]. It is not a text generator.

Image: v_i = normalize(P_I(VisionEncoder(I_i)))

Text: t_i = normalize(P_T(TextTransformer(T_i)[EOT]))

Here [EOT] is the contextualized end-of-text token representation, not a raw word embedding. Both v_i and t_i lie in R^D. The text Transformer in original CLIP was trained jointly with the image encoder, rather than reused as a pretrained GPT.

Contrastive learning: positive and negative pairs

For a batch of N matching image-text pairs, (I_i,T_i) is positive. The other N-1 texts are in-batch negatives for image I_i (and conversely for text T_i). Similarities form an N by N matrix; correct matches lie on its diagonal.

s_ij = (v_i dot t_j) / tau, tau > 0

L_image-to-text = -(1/N) sum_i log [exp(s_ii) / sum_j exp(s_ij)]

L_text-to-image = -(1/N) sum_i log [exp(s_ii) / sum_j exp(s_ji)]

L_CLIP = 0.5 * (L_image-to-text + L_text-to-image)

The temperature tau controls the sharpness of the softmax; CLIP learns a logit scale. Cross-entropy gradients flow through both encoders. This is not next-token prediction.

How pretrained CLIP is used

Zero-shot classification: supply an image and candidate prompts such as "a photo of a cat" or "a photo of a dog". Embed all prompts, score their cosine similarity with the image, and select the highest-scoring prompt. Softmax over these candidates gives relative scores, not calibrated real-world confidence.

predicted_class = argmax_j [cosine(f_I(image), f_T(prompt_j))]

Text-to-image retrieval: embed a query sentence and rank existing image embeddings. Image-to-image retrieval: compare image embeddings. CLIP cannot synthesize a new image or freely write a caption by itself.

4. From CLIP to generative VLMs

A generative VLM adds a language decoder that predicts text tokens conditioned on image information. Instead of scoring a supplied text, it learns a distribution over possible answers. Many architectures exist: visual-token projection, cross-attention, and other multimodal fusion strategies.

Visual tokens: Z = VisionEncoder(I) in R^(M x d_v)

Projected tokens: H = Projector(Z) in R^(M x d_llm)

p(Y | I,Q) = product_k p(y_k | y_1,...,y_(k-1), H, Q)

M is the number of visual tokens and Q is the user question. A projector can be linear or an MLP; it maps feature dimensions rather than reconstructing pixels. The autoregressive decoder generates text token by token.

5. LLaVA: Large Language and Vision Assistant

The original LLaVA work (Visual Instruction Tuning, 2023) connected a pretrained CLIP visual encoder to a pretrained language model through a trainable projection and used visual instruction data to teach the model to answer questions about images [3].

Simplified computation

Image I -> CLIP vision encoder -> visual features Z

Z -> trainable projector -> language-compatible visual tokens H

Concatenate / combine H with text prompt tokens -> autoregressive LLM -> answer

Training objectives and stages

Feature alignment: train a projector to connect pretrained vision features to the LLM embedding space using image-text supervision. Visual instruction tuning: train on image-conditioned questions and answers using autoregressive token cross-entropy. The original LLaVA recipe freezes or unfreezes components differently across stages; not every LLaVA version follows identical settings.

L_generation = - sum_k log p_theta(y_k | y_<k, image, instruction)

The loss supervises generated answer tokens; the model does not require the user to supply candidate captions at inference time. Example: image + "Describe this figure" -> a generated paragraph.

6. Direct comparison: CLIP vs. LLaVA

Question	CLIP	LLaVA
What is learned?	Joint image-text embedding alignment	Image-conditioned language generation
What is trained?	Image encoder + text encoder + projections	Vision-to-LLM alignment and instruction following
Loss	Symmetric contrastive cross-entropy	Autoregressive language cross-entropy
Input at inference	Image plus text candidates / corpus	Image plus open-ended question
Output	Similarity / ranked matches	Newly generated sentence or answer
Main limitation	Cannot directly generate descriptions	Can hallucinate or misread visual detail

7. Practical PyTorch mental models

CLIP: image_features = normalize(image_encoder(images)); text_features = normalize(text_encoder(token_ids)); logits = image_features @ text_features.T / temperature; labels = arange(B); loss = (CE(logits,labels) + CE(logits.T,labels))/2.

LLaVA: visual_tokens = projector(vision_encoder(images)); model_inputs = combine(visual_tokens, prompt_embeddings); logits = llm(model_inputs); loss = CE(answer_logits, answer_token_ids). These are conceptual sketches, not drop-in implementations.

Suggested exercise 1: use a pretrained CLIP model to rank five prompts for ten test images; inspect incorrect predictions and prompt sensitivity. Exercise 2: freeze a small vision encoder and train a toy caption decoder on paired images and short descriptions. Exercise 3: evaluate figure captioning separately from exact chart-value extraction.

8. Limits, evaluation, and common misconceptions

CLIP zero-shot performance depends on prompt wording, distribution shift, and training data. In-batch negatives can include semantically related descriptions. LLaVA may produce plausible but incorrect captions, counts, relationships, or chart readings. For scientific figures, verify axis labels and numerical claims against source data.

Not the same: VLM is the umbrella category; CLIP is a contrastive VLM; LLaVA is a generative VLM. A text Transformer is a neural network. CLIP text features are contextualized sentence embeddings, not generated captions. Neither original CLIP nor original LLaVA is a text-to-image generator.

9. Recommended reading (suggested order)

[4] Vinyals et al. (2015). Show and Tell: A Neural Image Caption Generator.

https://openaccess.thecvf.com/content_cvpr_2015/html/Vinyals_Show_and_Tell_2015_CVPR_paper.html

Early image-to-text generation: CNN + recurrent decoder.

[1] Radford et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP).

https://arxiv.org/abs/2103.00020

Read the contrastive objective, training setup, and zero-shot evaluation.

[2] Li et al. (2022). BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation.

https://arxiv.org/abs/2201.12086

Bridge between representation learning and generation.

[3] Liu et al. (2023). Visual Instruction Tuning (LLaVA).

https://arxiv.org/abs/2304.08485

Read the architecture, feature alignment, and instruction tuning.

[5] Xu et al. (2015). Show, Attend and Tell: Neural Image Caption Generation with Visual Attention.

https://proceedings.mlr.press/v37/xuc15.html

Early visual attention for caption generation.

10. Quick self-check

1. Why can CLIP classify unseen categories without training a new classification head? 2. What are the positive and negative pairs in CLIP? 3. Why is the EOT hidden state different from a raw token embedding? 4. Why does LLaVA need a projector? 5. Why can LLaVA describe a figure but CLIP cannot generate that description?

One-sentence takeaway: CLIP aligns vision and language in an embedding space; generative VLMs such as LLaVA use visual features to condition a language decoder and produce new text.